

PadChest

XML et notions avancées de bases de données

3ème Bachelier en informatique, orientation développement d'applications

Version 1 — année académique 2026-2027

1. Introduction

Vous trouverez dans ce document le contexte général du système informatique qui devra être réalisé, puis le détail des fonctionnalités demandées. Ce document peut donc être considéré comme votre « contrat de travail » : il permet de mieux appréhender l'organisation du travail de l'année et sa planification au travers des différentes semaines.

Les modalités d'évaluation se trouvent à la fin de cet énoncé, avec les explications quant à la construction de la cote finale. Il convient d'en prendre connaissance : elles vous engagent pour toute la durée de l'année scolaire.

Les données proviennent de la publication PadChest (<https://bimcv.cipf.es/bimcv-projects/padchest/>).

2. Contexte

Votre agent spécial a réussi à voler une partie des données concernant des images radiographiques thoraciques (en remplissant le formulaire de demande et en téléchargeant via leur site FTP — on les remercie, d'ailleurs). Mais pour que ce soit utilisable, il leur faut désormais structurer ces données (ils optent judicieusement pour le format XML), effectuer les transformations nécessaires pour pouvoir les présenter sur leur site web (grâce à XSLT), ainsi que les intégrer dans une base de données pour y rechercher des informations (à l'aide de XQuery).

2.1 Les images

La signalétique d'une image comporte tout un ensemble de métadonnées associées à cette image : son ID, l'ID du patient, le temps d'exposition, la position du patient, etc. Chaque image est un enregistrement stocké dans un fichier texte au format CSV, PADCHEST_chest_x_ray_images_labels_160K_01.02.19.csv, qui comporte 36 champs par film.

L'en-tête du fichier est le suivant. Remarquez que le tout premier champ n'a pas de nom : il s'agit de l'index de la ligne.

```
, ImageID, ImageDir, StudyDate_DICOM, StudyID, PatientID, PatientBirth,
PatientSex_DICOM, ViewPosition_DICOM, Projection, MethodProjection, Pediatric,
Modality_DICOM, Manufacturer_DICOM, PhotometricInterpretation_DICOM,
PixelRepresentation_DICOM, PixelAspectRatio_DICOM, SpatialResolution_DICOM,
BitsStored_DICOM, WindowCenter_DICOM, WindowWidth_DICOM, Rows_DICOM, Columns_DICOM,
XRayTubeCurrent_DICOM, Exposure_DICOM, ExposureInuAs_DICOM, ExposureTime,
RelativeXRayExposure_DICOM, ReportID, Report, MethodLabel, Labels, Localizations,
LabelsLocalizationsBySentence, labelCUIIS, LocalizationsCUIIS
```

Le séparateur de champ est la virgule (« , ») et certains champs peuvent avoir plusieurs valeurs. Dans ce cas, chaque valeur est contenue dans un tableau, lui-même entouré de guillemets. Le champ LabelsLocalizationsBySentence est encore plus spécial car il s'agit d'un tableau de tableaux.

Par exemple, pour l'enregistrement d'index 1, on a l'enregistrement ci-dessous dans le fichier texte :

```
1,135803415504923515076821959678074435083_fzis7d.png,0,20150914,
135803415504923515076821959678074435083,
313572750430997347502932654319389875966,1929.0,M,LATERAL,L,Manual review of
DICOM fields,No,CR,ImagingDynamicsCompanyLtd,MONOCHROME2,0,None,None,12,2631,
2065,3296,3236,400,10,10000,25,None,4991845, cambi pulmonar cronic sever .
sign fibrosis bibasal . sutil infiltr pseudonodul milimetr vidri deslustr
localiz bas . cifosis sever .,Physician,"['pulmonary fibrosis', 'chronic
changes', 'kyphosis', 'pseudonodule', 'ground glass pattern']","['loc basal',
'loc basal bilateral']","['pulmonary fibrosis', 'loc basal bilateral'],
['chronic changes'], ['kyphosis'], ['pseudonodule', 'ground glass pattern',
'loc basal']","['C0034069' 'C0742362' 'C2115817' 'C3544344'],['C1282378']
```

Vous devez seulement considérer les champs suivants :

Champ	Description
ImageID	Identifiant de l'image
ImageDir	Dossier zip contenant l'image
StudyID	Identifiant de l'étude
PatientID	Code du patient
PatientBirth	Année de naissance, format YYYY
PatientSex_DICOM	Sexe du patient
Projection	Classification parmi les 5 projections principales : PA (postéro-antérieur standard), L (latéral), AP (antéro-postérieur érigé ou vertical), AP_horizontal (antéro-postérieur horizontal), COSTAL (vues costales)
Pediatric	PED si l'acquisition a suivi un protocole pédiatrique
MethodProjection	Méthode appliquée pour assigner le type de projection, sur base des champs DICOM ou, à défaut, sur la sortie de classification d'un ResNet50 pré-entraîné
ReportID	Identifiant entier du rapport
Report	Extrait de texte du rapport original contenant l'interprétation radiographique. Le texte est prétraité : les mots sont racinisés et tokenisés. Chaque phrase est séparée par un « . »
MethodLabel	Méthode de labellisation : manuellement par des médecins (Physician) ou supervisée (RNN_model)
Labels	Séquence de labels uniques extraits du rapport
Localizations	Séquence de localisations anatomiques uniques extraites du rapport. Chaque localisation est toujours précédée du token loc
LabelsLocalizationsBySentence	Séquences de labels suivis de leurs localisations anatomiques, une sous-séquence par phrase, ordonnées selon l'ordre des phrases dans le rapport
labelCUIs	Séquence de CUIs du métathésaurus UMLS correspondant aux labels
LocalizationsCUIs	Séquence de CUIs du métathésaurus UMLS correspondant aux localisations

2.2 Avertissement : les données sont sales

Vous travaillez sur des données réelles, pas sur un jeu d'exercice. Le fichier fait environ 95 Mo et contient plus de 160 000 enregistrements. Il contient un certain nombre de pièges que vous devrez identifier vous-mêmes et traiter proprement.

Trois indices, parce que je suis gentil :

1. **Le nombre de lignes physiques du fichier n'est pas égal au nombre d'enregistrements.** Un programme qui lit « une ligne = un enregistrement » produira des résultats faux. À vous de comprendre pourquoi.
2. Certains champs contiennent des **valeurs manquantes ou aberrantes**. Vous documenterez votre décision de modélisation pour chacune : élément absent ? élément vide ? valeur par défaut ? enregistrement rejeté ?
3. La liste des valeurs possibles d'un champ **n'est pas nécessairement celle annoncée** dans le tableau ci-dessus. Vérifiez avant d'écrire une énumération dans votre DTD ou votre XSD.

Un rendu dont les statistiques finales sont fausses parce que le CSV a été mal parsé perd les points correspondants, même si le reste de la chaîne fonctionne.

3. Réalisations attendues

Vous utiliserez le fichier CSV comme base pour vos données, et effectuerez les opérations suivantes pour obtenir le site web souhaité.

3.1 Conversion vers un document XML

En première partie, les données du fichier seront converties en un fichier XML.

Le langage de programmation pour cette opération est laissé à votre appréciation parmi les possibilités suivantes : **(PL/)SQL, Java, C, C++, C#**.

Note. Attention aux API blackbox (que vous aurez le temps d'utiliser plus tard) et qui génèrent parfois des erreurs bien difficiles à déboguer. Préférez une approche « gestion de chaîne de caractères », qui est certes plus bas niveau mais sur laquelle vous avez le contrôle total et qui, finalement, ira plus vite à développer.

Vous êtes libres dans le nom des tags, dans le choix des types d'éléments (avec données ou simple tag), d'attributs, etc. Mais votre structure doit respecter les contraintes suivantes :

- **Au moins trois niveaux d'imbrication significatifs.** Une conversion « une ligne CSV = un élément plat » est refusée. En particulier : un patient possède une ou plusieurs études, une étude possède une ou plusieurs images. Cette hiérarchie doit apparaître dans votre document, et les informations du patient ne doivent donc pas être répétées à chaque image.
- **Au moins un attribut** dans au moins un élément.
- **Au moins une référence interne** (ID/IDREF en DTD) : à vous de trouver où c'est pertinent.
- **Un encodage UTF-8** avec la déclaration XML correspondante, et un **échappement correct** : votre convertisseur ne doit pas produire de document mal formé si une donnée contient &, < ou ", même si ce cas ne se présente pas forcément dans ce fichier-ci.

Le regroupement demandé implique que vous ne pouvez pas vous contenter d'écrire au fil de la lecture. Réfléchissez à la structure de données intermédiaire, et à ce qu'elle coûte en mémoire sur 160 000 enregistrements.

3.2 Structure du document XML

La structure de votre document devra être déterminée. C'est dans cette partie que vous validerez formellement les choix que vous avez effectués dans la section précédente.

- **Au minimum** : vous réaliserez un document DTD précisant la structure.
- **Pour les pros** : vous réaliserez, en plus du DTD, un document XSD précisant la structure (types simples dérivés, énumérations, restrictions de cardinalité, key/keyref).

Note. Ces structures doivent impérativement être écrites à la main, et pas générées automatiquement à partir d'un outil. Elles doivent aussi être restrictives : un schéma qui accepte n'importe quoi (ANY, xs:string partout) ne vaut pas grand-chose et sera coté comme tel.

3.3 Validation du fichier XML et calculs

Une fois votre document XML et votre document de structure en main, il vous faudra vérifier si celui-ci est bien valide. Pour ce faire, il faudra réaliser un parser. Celui-ci sera écrit en langage Java.

- **Au minimum** : vous utiliserez un parser SAX.
- **Pour les pros** : vous utiliserez un parser DOM en plus du SAX.
- **Pour les experts** : ajoutez une troisième implémentation avec StAX et évaluez les performances de vos programmes en comparant les temps d'exécution ainsi que la consommation mémoire (y compris, le cas échéant, en comparant aussi une validation avec DTD et une validation avec XSD). Vos mesures seront présentées dans un petit rapport précisant la méthodologie employée : JVM, options -Xmx, nombre de répétitions, machine.

Un des trois parsers ne parviendra probablement pas à traiter le fichier complet avec les réglages par défaut de la JVM. Ce n'est pas un bug de votre code : c'est précisément le phénomène que je veux vous voir expliquer.

Dans tous les cas : en plus de vérifier la validité du fichier, vous effectuerez via le parser les calculs suivants.

1. Compter le nombre d'images qui contiennent la localisation loc right.
2. Classer les labels qui apparaissent le plus souvent et afficher la liste des 10 labels les plus fréquents et, pour chacun, combien de fois il apparaît.
3. Compter le nombre de patients distincts et, pour chaque valeur de Projection, le nombre d'images correspondantes.

Les résultats seront affichés par le programme lui-même, et non recopiés à la main dans le rapport.

3.4 Création du site web avec XSLT

Une fois votre fichier XML en votre possession, vous effectuerez une transformation à l'aide d'un fichier style sheet (XSLT, donc) afin de générer une page HTML affichant proprement les informations.

Changement important par rapport aux années précédentes — lisez ceci.

Les navigateurs sont en train de retirer le support de XSLT. Chrome coupe XSLT sur les versions stables le **17 novembre 2026** (Chrome 158) ; Firefox et WebKit ont annoncé la même intention. L'astuce classique consistant à ajouter une instruction `<?xml-stylesheet ?>` en tête du XML puis à ouvrir le fichier dans un navigateur ne fonctionnera plus pendant votre année. Ne construisez pas votre projet dessus.

Par ailleurs, les navigateurs n'ont jamais implémenté que XSLT 1.0 (1999). Les processeurs modernes vous donnent accès à XSLT 3.0, nettement plus agréable : `xsl:for-each-group`, `xsl:function`, formats de sortie multiples, sortie JSON, mode streaming.

Vous produirez donc votre HTML par une transformation hors navigateur, au choix :

- **Saxon-HE** (Java, gratuit, XSLT 3.0), en ligne de commande ou depuis un programme Java ;
- **xsltproc** / **libxslt** (XSLT 1.0) ;
- l'**extension XSL de PHP** si vous passez par un serveur Apache / XAMPP ;
- **SaxonJS** dans le navigateur, si vous tenez vraiment à une transformation côté client : c'est du JavaScript, ça survivra à la suppression de XSLT natif.

Le HTML généré est ensuite servi comme n'importe quelle page statique. Puisque nous travaillons avec des gens pressés qui veulent une solution rapide, un petit serveur Apache (par exemple avec XAMPP) fait très bien l'affaire.

- **Au minimum** : ce sera fonctionnel. Il sera possible de voir toutes les informations, dans une table par exemple.
- **Pour les pros** : on imaginera un document un peu structuré, joli, ergonomique, sans pour autant faire intervenir des technologies supplémentaires.
- **Pour les experts** : vous rajouterez des fonctionnalités à l'aide du CSS, du JavaScript, etc. Vous pouvez laisser libre cours à votre imagination et vos envies. Étonnez-moi 😊

3.5 Intégration dans une base de données XQuery

À l'aide du logiciel BaseX (version 12 ou supérieure), vous intégrerez votre fichier XML à l'intérieur d'une base de données XQuery. Vous écrirez au minimum les requêtes suivantes.

1. Compter le nombre d'images qui contiennent la localisation loc right.
2. Classer les labels les plus fréquents et afficher le top 10 avec, pour chacun, son nombre d'apparitions.
3. Pour un label donné en paramètre, la liste des patients concernés, triés par année de naissance.
4. La liste des paires (label, localisation) les plus fréquentes, extraites de LabelsLocalizationsBySentence.

Les résultats des requêtes 1 et 2 **doivent être identiques** à ceux de votre parser Java. Si ce n'est pas le cas, c'est un excellent sujet de discussion à la défense.

- **Au minimum** : ce sera fonctionnel (intégration et requêtes).
- **Pour les pros** : la base de données et le résultat des requêtes pourront être obtenus à l'aide d'un navigateur web, via le webservice RESTXQ de BaseX.
- **Pour les experts** : le webservice sera appelé par la page générée en XSLT (fetch / AJAX), et vous comparerez les temps de réponse avec et sans index BaseX (texte intégral, attributs). Vous pouvez également exposer une des réponses en JSON (fn:xml-to-json, annotations RESTXQ) et expliquer dans quel cas c'est préférable.

4. Modalités de rendu

Ce projet est à réaliser **par binôme**. Il sera à présenter pendant l'année, pendant la séance de laboratoire. Je vous recommande de le faire après le congé d'automne, et de ne pas attendre la dernière semaine.

Dans tous les cas, pour pouvoir présenter le projet, il faut l'avoir envoyé par mail à **samuel.hiard@hepl.be** au plus tard le vendredi précédant le jour du labo, avec 23h59 comme heure limite.

Le rendu contient :

NOM1_NOM2/	
├ README.md	comment tout exécuter, décisions, note de synthèse
├ 1_conversion/	code source du convertisseur CSV -> XML
├ 2_structure/	DTD et/ou XSD (et Schematron)
├ 3_parsing/	parsers Java + résultats des calculs
├ 4_xslt/	feuille(s) de style + HTML généré + commande utilisée
├ 5_xquery/	requêtes XQuery (+ module RESTXQ le cas échéant)
└ extrait.xml	les 200 premiers enregistrements de votre XML (le fichier complet est trop lourd pour un mail)

Le README précise explicitement :

- la commande exacte pour rejouer chaque étape depuis zéro ;
- vos décisions de modélisation et leur justification (les fameux pièges du point 2.2) ;
- vos résultats numériques pour les calculs demandés ;
- la répartition du travail entre les deux membres du binôme ;
- l'usage éventuel d'assistants IA : lesquels, pour quoi, et ce que vous avez modifié ou corrigé dans ce qu'ils ont produit.

Sur ce dernier point, soyons clairs : utiliser un assistant IA n'est pas interdit et ne vous coûte aucun point. **Ne pas comprendre ce que vous rendez, si.**

À la défense, chaque membre du binôme devra expliquer une partie du code écrite par l'autre, et effectuer une petite modification en direct : ajouter un champ, changer une contrainte du schéma, modifier une requête. Un binôme incapable de faire cela ne peut pas obtenir les points de la partie concernée, quelle que soit la qualité du rendu.

5. Évaluation

L'évaluation sera séparée en deux catégories :

1. **Théorie.** Évaluée via un QCM. Compte pour 30 % des points.
2. **Laboratoire.** Évaluée via deux projets. Compte pour 70 % des points.

Afin d'évaluer la partie laboratoire, deux projets seront à réaliser :

1. ce projet XML, qui compte pour 50 % de la note de labo ;
2. un autre projet, plutôt orienté SGBD, dont l'énoncé sera disponible ultérieurement, et qui comptera pour 50 % de la note de labo.

Attention, toutes les moyennes sont géométriques.

Comme vous savez calculer, vous aurez compris que ce projet compte donc pour 30 % de la note. Les points sont répartis de la manière suivante :

Partie	Points
Conversion vers le document XML	5
Réalisation de la DTD et/ou du XSD	4
Parsing et calculs statistiques	6
Site web avec XSLT	5
XQuery	5
Total	25

Une pénalité pouvant aller jusqu'à **-5** s'applique en cas de rendu non conforme aux modalités du point 4 : arborescence, README incomplet, résultats non reproductibles.

Pour certaines réalisations, il est fait mention de « au minimum », « pour les pros » et « pour les experts ». La signification est la suivante :

- **Au minimum** : la base. Ce que j'attends vraiment au minimum du minimum. Arrêtez-vous là si vous visez la réussite simple — ce que vous aurez, mais seulement si c'est bien réalisé.
- **Pour les pros** : une réalisation un peu plus pro ou demandant un peu plus d'implication et de travail. Ne sera évalué que si l'item « au minimum » est réalisé. Pour ceux qui visent la distinction.
- **Pour les experts** : pour ceux que le sujet passionne et qui veulent vraiment se spécialiser dans cette discipline. Ou pour ceux qui visent l'excellence et la plus grande distinction.

Il y a beaucoup à faire, donc ne traînez pas.

Bon courage et, surtout, amusez-vous bien 😊